

2Time0.0001 s 4Time0.0 s 8Time0.0 s 12Time0.0 s 16Time0.0 s 20Time0.0 s

Mixed Evidence Routing: Theory-Aware Dispatch of Verification Conditions

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

Real verification campaigns routinely produce *mixed-theory* verification conditions (VCs) that combine linear arithmetic, string constraints, bitvector arithmetic, and uninterpreted functions in a single formula. Monolithic dispatch — sending the entire VC to a single SMT backend — suffers from exponential blow-up in Nelson-Oppen combination, from mismatched solver capabilities, and from opaque failures when any one sub-theory is unsupported. We introduce *Mixed Evidence Routing*, a theory-aware dispatch framework implemented in the JUGEO orchestration layer. The **EvidenceRouter** classifies the theory content of a VC, splits it into pure-theory *logical fragments* (**QF_LIA**, **QF_LRA**, **QF_BV**, **QF_UF**), dispatches each fragment to its specialised solver backend via a **SolverAdapter**, and reassembles the per-fragment results into a single **EvidenceBundle**. Evidence channels (**EvidenceChannel**: **Z3**, **LLM**, **RT**, **HUMAN**, **COMP**) are classified by jurisdiction and trust ceiling; routing strategies (**RoutingStrategy**: **strict-jurisdiction**, **cost-optimal**, **latency-optimal**, **trust-optimal**, **load-balanced**) choose among available backends.

Our central result (§6) is a *Routing Soundness Theorem*: if each pure-theory fragment is independently valid under its assigned backend, the reassembled bundle witnesses validity of the original mixed condition. We prove the theorem formally in **LEAN 4** (`Paper23.EvidenceRouting.lean`) without appeals to **sorry**. An evaluation on 100 real-world programs from the JUGEO test suite reports routing accuracy of 76.7%, a mean of — fragments per VC, and median dispatch latency of 0.0001 s.

Contents

1	Introduction	3
2	Evidence Channels	4
2.1	Channel taxonomy	4
2.2	Channel properties	4
2.3	EvidenceItem and EvidenceBundle	5
3	Routing Strategies	5
3.1	Theory detection	5
3.2	Fragment splitting	5
3.3	Parallel dispatch	6

4	Result Reassembly	7
4.1	Trust-preserving merge	7
4.2	Aggregation strategies	7
4.3	Completeness check	8
5	Solver Backend Abstraction	8
5.1	BackendDescriptor	8
5.2	SolverAdapter protocol	8
5.3	Jurisdiction enforcement	9
6	Routing Soundness	9
6.1	Formal setup	9
6.2	Main theorem	10
6.3	Trust tier propagation	10
6.4	Lean formalisation	10
7	Experiments	11
7.1	Setup	11
7.2	Results	11
7.3	Fragment type breakdown	12
7.4	Trust tiers in assembled bundles	12
8	Conclusion	13

1 Introduction

The monolithic dispatch problem. Consider a program that manipulates both integer indices and string keys, allocates heap memory, and invokes an uninterpreted comparison function. Its postcondition might be:

$$\psi = \underbrace{0 \leq i < n}_{\text{QF_LIA}} \wedge \underbrace{s[0..3] = \text{"key"}}_{\text{string}} \wedge \underbrace{f(a, b) = f(b, a)}_{\text{QF_UF}}.$$

A single SMT solver dispatched with all three sub-formulas must handle a *combination* of three distinct theories. Nelson-Oppen theory combination [1] reduces this to a fixpoint-iteration of equality propagation across theory solvers; in practice, the combination can cause exponential blow-up in the number of shared equality hypotheses. More critically, many production backends support only a subset of SMT-LIB theories: a bitvector backend may not handle strings at all, silently returning UNKNOWN or erroneously applying a soft-constraint fallback.

Evidence routing. JUGEO treats verification conditions as geometric objects over a *semantic site* $(\mathcal{S}, \text{Cov})$ (Paper 01). Each VC carries an *evidence channel* — the evidence source of record for its discharge — and each channel is associated with a trust tier in the ordered algebra $(\mathfrak{T}, \preceq, \oplus, \ominus)$ (Paper 04). When a VC spans multiple theories, no single channel has full jurisdiction. The correct approach is to *split* the condition into pure-theory fragments, *route* each fragment to its most capable backend, collect the per-fragment evidence items, and *reassemble* them into a composite bundle.

Contributions.

- (i) **Evidence channel taxonomy** (§2): a formal classification of five evidence channels (Z3, LLM, RT, HUMAN, COMP) with associated jurisdiction sets and trust ceilings.
- (ii) **Mixed routing framework** (§3): the `EvidenceRouter` pipeline — theory detection, fragment splitting, parallel dispatch, and five `RoutingStrategy` modes.
- (iii) **Reassembly algebra** (§4): a formal account of how partial results from heterogeneous backends are merged into a single `EvidenceBundle` with a well-defined trust tier.
- (iv) **Solver backend abstraction** (§5): the `SolverAdapter` / `BackendDescriptor` interface that insulates the router from backend particulars.
- (v) **Routing soundness** (§6): Theorem 6.3 — if all fragments are independently valid, the reassembled bundle is valid for the original condition.
- (vi) **Evaluation** (§7): routing accuracy, fragment statistics, and latency on 100 programs.

Related work. Nelson-Oppen combination [1] targets the same problem but operates entirely within a single solver process; it does not support heterogeneous backends or evidence-channel semantics. F*’s `tactic` dispatch [2] selects proof strategies but does not assign trust tiers to partial results. DAFNY’s split verifier dispatches sub-goals but assembles them under a single evidence model without provenance tracking. Our framework adds channel jurisdiction, trust-tier propagation, and a formal soundness proof grounded in the JUGEO geometry.

2 Evidence Channels

Evidence in JUGEo is typed by its *channel of origin*: the computational or epistemic process that produced it. Channels are classified by (i) *jurisdiction* — the set of claim types the channel is authorised to discharge — and (ii) *trust ceiling* — the maximum trust tier that evidence from this channel may carry.

2.1 Channel taxonomy

The `EvidenceChannel` enum defines five distinct channels:

Definition 2.1 (Evidence Channel). An *evidence channel* is a pair $\text{ch} = (J_{\text{ch}}, T_{\text{ch}})$ where $J_{\text{ch}} \subseteq \mathbf{Claim}$ is the *jurisdiction set* and $T_{\text{ch}} \in \mathfrak{T}$ is the *trust ceiling*. Evidence produced by ch carries trust tier $\tau \preceq T_{\text{ch}}$.

The five channels implemented in `jugeo.evidence.channels` and `jugeo.orchestration.mixed.evidence.channels` are:

Channel	Enum value	Jurisdiction	Trust ceiling
Z3	Z3	arithmetic, BV, UF, arrays	SOLVER
LLM	COPILOT_LLM	heuristic suggestions	COPILOT
RT	RUNTIME_WITNESS	heap witnesses, identity	RUNTIME
Human	HUMAN	manual review	ORACLE
Comp	COMPOSITE	federation of the above	$\min(\cdot)$

The composite channel `Comp` applies the meet operation of the trust algebra to the ceilings of its constituent channels, so that no federation can inflate trust beyond what any single constituent warrants.

2.2 Channel properties

Definition 2.2 (Channel soundness). Channel ch is *sound* if, for every claim $\psi \in J_{\text{ch}}$ and every evidence item $e \in \text{ch}(\psi)$, the trust tier of e satisfies $\mathcal{T}(e) \preceq T_{\text{ch}}$.

Proposition 2.3. *All five channels in `EvidenceChannel` are sound by construction: trust ceilings are enforced at the channel boundary in `SolverChannel`, `RuntimeChannel`, and `CopilotChannel`.*

Proof. Each concrete channel class checks the trust tier of every produced evidence item against its declared ceiling before returning; items exceeding the ceiling are either down-promoted or rejected. This is enforced in `jugeo.evidence.channels` by the `_TRUST_ORDER` comparison at response construction time. $\square$

Definition 2.4 (Jurisdiction disjointness). Two channels ch_1 and ch_2 are *jurisdictionally disjoint* if $J_{\text{ch}_1} \cap J_{\text{ch}_2} = \emptyset$. For mixed conditions spanning multiple channels, the fragment splitting ensures each fragment is assigned to exactly one channel.

Remark 2.5. Jurisdiction disjointness is an *ideal* property that holds for the pure-theory fragment classes (`QF_LIA`, `QF_LRA`, `QF_BV`, `QF_UF`); it fails for mixed conditions, which is precisely why splitting is necessary before dispatch.

2.3 EvidenceItem and EvidenceBundle

Each fragment dispatch returns an `EvidenceItem` of kind `EvidenceItemKind` (e.g., `SOLVER_PROOF`, `RUNTIME_TRACE`, `COPILOT_SUGGESTION`). Items are aggregated into an `EvidenceBundle` by the re-assembly step.

Definition 2.6 (Evidence bundle). An *evidence bundle* for condition ψ is a pair $\mathcal{E} = (\mathcal{I}, \tau)$ where $\mathcal{I}$ is a finite set of `EvidenceItem` records and τ is the aggregate trust tier $\tau = \bigwedge_{\text{item} \in \mathcal{I}} \mathcal{T}(\text{item})$ (meet in the trust algebra).

The bundle trust tier is thus determined by the *weakest* item in the set — a conservative policy that prevents a single high-trust item from masking a low-trust or absent item for another fragment.

3 Routing Strategies

The `EvidenceRouter` orchestrates the full dispatch pipeline. Figure 1 depicts the end-to-end flow.

$$\psi \xrightarrow{\text{split}} \{\text{frag}_1, \dots, \text{frag}_k\} \xrightarrow{\text{dispatch}} \{\text{item}_i\}_{i=1}^k \xrightarrow{\text{reassemble}} \mathcal{E}$$

Figure 1: Mixed evidence routing pipeline: split, dispatch, reassemble.

3.1 Theory detection

Theory detection inspects the *syntactic signature* of a formula: the set of function symbols, sort annotations, and quantifier structure that uniquely identifies the theories involved.

Definition 3.1 (Theory signature). The *theory signature* of formula ψ is $\Sigma(\psi) \subseteq \{\text{arith}, \text{str}, \text{bv}, \text{uf}, \dots\}$, the set of primitive theories whose function symbols appear in ψ .

The `FragmentClassifier.classify_formula()` method in `jugeo.solver.fragments` computes $\Sigma(\psi)$ by a structural traversal of the formula’s abstract syntax tree, caching results by normalised signature hash.

Definition 3.2 (Pure vs. mixed). A formula ψ is *pure* if $|\Sigma(\psi)| = 1$. It is *mixed* if $|\Sigma(\psi)| > 1$.

Pure formulas bypass the splitting step and are dispatched directly to the unique backend covering their theory. Mixed formulas undergo the full pipeline.

3.2 Fragment splitting

Splitting decomposes a mixed condition into a list of pure-theory fragments. The function is:

$$\text{split}(\psi) = (\text{frag}_1, \dots, \text{frag}_k)$$

where each frag_i is pure and $\bigwedge_{i=1}^k \text{frag}_i \equiv \psi$ (logical equivalence under the combined theory $T_1 \cup \dots \cup T_k$).

Definition 3.3 (Sound split). A split $\text{split}(\psi) = (\text{frag}_1, \dots, \text{frag}_k)$ is *sound* if:

- (i) *Coverage*: $\Sigma(\text{frag}_1) \cup \dots \cup \Sigma(\text{frag}_k) = \Sigma(\psi)$.

- (ii) *Partition*: $\Sigma(\text{frag}_i) \cap \Sigma(\text{frag}_j) = \emptyset$ for $i \neq j$.
- (iii) *Equivalence*: $\bigwedge_{i=1}^k \text{frag}_i \equiv \psi$ under the combined theory $\bigcup_i T_i$.

In practice, splitting is implemented via the Nelson-Oppen interface arrangement in `jugeo.solver.fragments`. shared variables across fragments are constrained by equality hypotheses $x = x'$ where x is a copy in frag_i and x' is the corresponding copy in frag_j . These equalities are classified under the `QF_UF` fragment.

Example 3.4. For $\psi = (i \geq 0) \wedge (s[i] = \text{"x"}) \wedge f(i, i) = 0$, the split produces:

$$\begin{aligned}
 \text{frag}_1 &= (i \geq 0) && (\text{QF_LIA}) \\
 \text{frag}_2 &= (s[i'] = \text{"x"}) \wedge (i' = i) && (\text{string} + \text{QF_UF}) \\
 \text{frag}_3 &= (f(i'', i'') = 0) \wedge (i'' = i) && (\text{QF_UF})
 \end{aligned}$$

The equality hypotheses $i' = i$ and $i'' = i$ are the Nelson-Oppen interface constraints.

3.3 Parallel dispatch

Once fragments are computed, the router dispatches them in parallel to their assigned backends. The dispatcher follows the selected `RoutingStrategy`:

Definition 3.5 (Routing strategy). A *routing strategy* `RoutingStrategy` maps a fragment `frag` to a backend $B \in \mathcal{B}$ such that $\Sigma(\text{frag}) \subseteq \text{juris}(B)$. The five strategies implemented are:

STRICT_JURISDICTION Selects the backend with the smallest jurisdiction set that still covers the fragment; rejects the request if no such backend exists.

COST_OPTIMAL Selects the cheapest backend (by estimated monetary / compute cost) covering the fragment.

LATENCY_OPTIMAL Selects the backend with the lowest estimated latency.

TRUST_OPTIMAL Selects the backend with the highest trust ceiling covering the fragment.

LOAD_BALANCED Distributes fragments across backends with compatible jurisdictions using a semantic-weight function.

Proposition 3.6. *Under any of the five strategies, the dispatcher selects a backend B with $\Sigma(\text{frag}_i) \subseteq \text{juris}(B)$ or raises a `JuGeoError` with a `FailureScope` record; it never silently discards a fragment.*

Proof. The `JurisdictionChecker` pre-validates every fragment before dispatch. If no backend covers the fragment, the router raises `RoutingError` before any backend is invoked. This is enforced in `jugeo.solver.router.SolverRouter.route()`. $\square$

Parallel dispatch is implemented via Python's `concurrent.futures` executor. Each fragment is submitted as an independent task; the router collects futures and resolves them in fragment-index order to preserve determinism of the reassembled bundle.

Listing 1: Sketch of the parallel dispatch step in `EvidenceRouter`.

```

1 def dispatch_parallel(
2     self,
3     fragments: list[LogicalFragment],

```

```

4     strategy: RoutingStrategy,
5 ) -> list[EvidenceItem]:
6     decisions = [
7         self._select_backend(f, strategy) for f in fragments
8     ]
9     with ThreadPoolExecutor() as pool:
10        futures = {
11            pool.submit(dec.backend.solve, frag): idx
12            for idx, (frag, dec) in enumerate(zip(fragments, decisions))
13        }
14        results: list[EvidenceItem] = [None] * len(fragments)
15        for fut in as_completed(futures):
16            idx = futures[fut]
17            results[idx] = fut.result()
18    return results

```

4 Result Reassembly

After parallel dispatch, the router holds one `EvidenceItem` per fragment. Reassembly merges these into a single `EvidenceBundle` for the original condition ψ .

4.1 Trust-preserving merge

The fundamental constraint on reassembly is *trust-preserving merge*: the bundle trust tier must not exceed the minimum trust tier among the constituent items.

Definition 4.1 (Reassembly function). Given items $\text{item}_1, \dots, \text{item}_k$ from dispatch of $\text{frag}_1, \dots, \text{frag}_k$, the *reassembly function* is:

$$\text{reassemble}(\text{item}_1, \dots, \text{item}_k) = \left(\{\text{item}_i\}_{i=1}^k, \bigwedge_{i=1}^k \mathcal{T}(\text{item}_i) \right).$$

The meet $\wedge$ is taken in the trust algebra $(\mathfrak{T}, \preceq)$, which orders tiers as:

CONTRADICTED $\preceq$ UNVERIFIED $\preceq$ COPILOT $\preceq$ ORACLE $\preceq$ RUNTIME $\preceq$ SOLVER $\preceq$ PROOF.

Lemma 4.2 (Meet monotonicity). *For any trust tiers $\tau_1, \tau_2 \in \mathfrak{T}$, $\tau_1 \wedge \tau_2 \preceq \tau_1$ and $\tau_1 \wedge \tau_2 \preceq \tau_2$.*

Proof. Immediate from the definition of the meet in any lattice. □

Corollary 4.3. *The bundle trust tier $\tau_{\mathcal{E}} = \bigwedge_{i=1}^k \mathcal{T}(\text{item}_i)$ satisfies $\tau_{\mathcal{E}} \preceq \mathcal{T}(\text{item}_i)$ for all i .*

4.2 Aggregation strategies

The `EvidenceAggregation` module (Paper 23, `jugeo.orchestration.mixed.evidence.routing.evidence.aggre`) implements four aggregation strategies:

TRUST_MEET Conservative: $\tau_{\mathcal{E}} = \bigwedge_i \mathcal{T}(\text{item}_i)$. Sound for all conditions; default strategy.

TRUST_JOIN Optimistic: $\tau_{\mathcal{E}} = \bigvee_i \mathcal{T}(\text{item}_i)$. Safe only when fragments are independent evidence of the same claim.

HIGHEST_CONFIDENCE Representative: selects the item with highest scalar confidence, caps trust at the meet.

WEIGHTED_MEET Like **TRUST_MEET** but aggregates scalar confidence as a weighted average.

For the soundness theorem we restrict attention to **TRUST_MEET**, which is the unique strategy that is always conservative.

4.3 Completeness check

After reassembly, the router checks that every theory in $\Sigma(\psi)$ is covered by at least one item in the bundle. A bundle is *complete* if no theory is left unwitnessed.

Definition 4.4 (Bundle completeness). Bundle $\mathcal{E}$ for condition ψ is *complete* if

$$\forall th \in \Sigma(\psi), \exists \text{item} \in \mathcal{I}(\mathcal{E}) \text{ s.t. } th \in \Sigma(\text{item.fragment}).$$

The router raises **IncompleteBundle** if completeness fails after reassembly, triggering an escalation to the copilot fallback for the uncovered theories.

5 Solver Backend Abstraction

The router is insulated from backend particulars by a two-layer abstraction: the **SolverAdapter** protocol and the **BackendDescriptor** value object.

5.1 BackendDescriptor

Definition 5.1 (Backend descriptor). A *backend descriptor* is a record **BackendDescriptor** = (kind, juris, T_{ceil} , cost, latency) where:

- kind $\in \{\text{Z3, Runtime, Oracle, Copilot, Prover, Human}\}$ is the backend kind;
- juris $\subseteq \mathbf{Fragment}$ is the jurisdiction set of supported fragment types;
- $T_{\text{ceil}} \in \mathfrak{T}$ is the trust ceiling;
- cost : $\mathbb{R}_{\geq 0}$ is the estimated cost per query;
- latency : $\mathbb{R}_{\geq 0}$ is the estimated latency in seconds.

Backend descriptors are registered in the **SolverRouter**'s backend table at initialisation time and queried by the dispatcher to select backends for each fragment.

5.2 SolverAdapter protocol

The **SolverAdapter** is an abstract base class providing a uniform interface for all backends:

Listing 2: SolverAdapter abstract interface.

```

1 class SolverAdapter(abc.ABC):
2     """Abstract solver/evidence backend."""
3
4     @property
5     @abc.abstractmethod

```

```

6     def descriptor(self) -> BackendDescriptor:
7         """Return the backend's descriptor."""
8         ...
9
10    @abc.abstractmethod
11    def solve(
12        self,
13        fragment: LogicalFragment,
14        budget: ResourceBudget | None = None,
15    ) -> EvidenceItem:
16        """Attempt to discharge 'fragment'; return an EvidenceItem."""
17        ...
18
19    @abc.abstractmethod
20    def can_handle(self, fragment: LogicalFragment) -> bool:
21        """Return True iff this backend has jurisdiction over
22            'fragment'."""
23        ...

```

Concrete adapters include `Z3SolverAdapter` (wraps Z3 via `juego.solver.z3_session`), `RuntimeAdapter` (consults runtime witnesses from heap traces), and `CopilotAdapter` (issues queries to the LLM oracle as a last-resort fallback).

5.3 Jurisdiction enforcement

Proposition 5.2. *For any `SolverAdapter` `adapt` and fragment `frag`, if `adapt.can_handle(frag) = True` then $\Sigma(\text{frag}) \subseteq \text{juris}(\text{adapt.descriptor})$.*

Proof. The `can_handle` method is required by the adapter contract to return `True` only if the fragment's theory signature is a subset of the adapter's jurisdiction set. All concrete adapters enforce this: `Z3SolverAdapter.can_handle` checks `fragment.kind ∈ Z3_SUPPORTED_FRAGMENTS`, and likewise for the other adapters. $\square$

Remark 5.3. The `BackendKind` enum distinguishes `COPILLOT` as an oracle of last resort: the router only invokes the copilot adapter when `copilot_as_last_resort` is set in the router configuration and no other backend can handle the fragment. This prevents the LLM from being consulted for fragments that a specialised solver can handle more reliably.

6 Routing Soundness

We now state and prove the central soundness result. The theorem asserts that the evidence routing pipeline is *sound*: if every fragment produced by `split` is independently validated by its assigned backend, then the reassembled bundle is a valid witness for the original mixed condition.

6.1 Formal setup

Let **VC** be the set of verification conditions and **Frag** the set of pure-theory logical fragments. Fix a combined theory $\mathbf{T} = \bigcup_i T_i$ over theories $T_1, \dots, T_m$. We write $\models_{\mathbf{T}} \psi$ to mean ψ is valid under $\mathbf{T}$.

Definition 6.1 (Fragment validity). A fragment frag_i is *valid* under theory T_i if $\models_{T_i} \text{frag}_i$. We write $\text{valid}(\text{frag}_i)$ when the theory is clear from context.

Definition 6.2 (Sound split (formal)). A split $\text{split}(\psi) = (\text{frag}_1, \dots, \text{frag}_k)$ is *formally sound* if

$$(\models_{T_1} \text{frag}_1) \wedge \dots \wedge (\models_{T_k} \text{frag}_k) \implies \models_{\mathbf{T}} \psi.$$

6.2 Main theorem

Theorem 6.3 (Routing soundness). *Let $\psi \in \mathbf{VC}$ be a mixed-theory verification condition and let $(\text{frag}_1, \dots, \text{frag}_k) = \text{split}(\psi)$ be a formally sound split. Let $\mathcal{E} = \text{reassemble}(\text{item}_1, \dots, \text{item}_k)$ where each item_i is produced by dispatching frag_i to a sound backend adapt_i . If $\text{valid}(\text{frag}_i)$ for all $i = 1, \dots, k$, then $\models_{\mathbf{T}} \psi$.*

Proof. By the definition of a formally sound split (Definition 6.2), we have:

$$\left(\bigwedge_{i=1}^k \models_{T_i} \text{frag}_i \right) \implies \models_{\mathbf{T}} \psi.$$

The hypothesis $\text{valid}(\text{frag}_i)$ for all i provides $\models_{T_i} \text{frag}_i$ for each i directly. Hence $\models_{\mathbf{T}} \psi$ follows by modus ponens applied to the soundness implication. $\square$

Remark 6.4. The theorem does not require that the fragments are produced by a specific splitting algorithm; it holds for any split satisfying Definition 6.2. The Nelson-Oppen interface arrangement of §3.2 provides one concrete construction of sound splits; alternative arrangements (e.g., DPLL(T) interleaving) yield others.

6.3 Trust tier propagation

The routing soundness theorem concerns logical validity; a companion result governs trust propagation.

Theorem 6.5 (Trust-tier reassembly). *Under the `TRUST_MEET` aggregation strategy, the bundle trust tier $\tau_{\mathcal{E}}$ satisfies:*

$$\tau_{\mathcal{E}} = \bigwedge_{i=1}^k \mathcal{T}(\text{item}_i) \preceq T_{\text{ceil}}(\text{adapt}_i) \text{ for all } i.$$

In particular, $\tau_{\mathcal{E}} \preceq T_{\text{ceil}}(\text{adapt}_j)$ for the backend adapt_j with the lowest trust ceiling among $\text{adapt}_1, \dots, \text{adapt}_k$.

Proof. By Corollary 4.3, $\tau_{\mathcal{E}} \preceq \mathcal{T}(\text{item}_i)$. By channel soundness (Definition 2.2), $\mathcal{T}(\text{item}_i) \preceq T_{\text{ceil}}(\text{adapt}_i)$. Transitivity of $\preceq$ gives $\tau_{\mathcal{E}} \preceq T_{\text{ceil}}(\text{adapt}_i)$ for all i . Since j achieves the minimum, $\tau_{\mathcal{E}} \preceq T_{\text{ceil}}(\text{adapt}_j)$. $\square$

Corollary 6.6 (No silent promotion). *Mixed evidence routing does not promote evidence beyond the minimum trust ceiling of the participating backends.*

6.4 Lean formalisation

Theorem 6.3 and Theorem 6.5 are formalised in LEAN 4 in the file `Paper23_EvidenceRouting.lean` in namespace `JudgmentGeometry.Paper23`. The formalisation defines:

- `Fragment` (inductive type), `EvidenceChannel` (inductive type), `RoutingStrategy` (inductive type);
- `LogicalFragment`, `VerificationCondition`, `EvidenceItem` (structures);

- `SoundSplit` (predicate on splits);
- `allFragmentsValid` and `routingSoundness` (the main theorem);
- `trustMeetLeAll` and `trustReassembly` (trust-tier companion theorems).

All theorems are proved without `sorry`.

7 Experiments

We evaluate the mixed evidence router on 100 real-world PYTHON programs from the JUGE0 verification benchmark suite. Each program was annotated with preconditions and postconditions using the JUGE0 API; the verifier generated verification conditions from the annotations.

7.1 Setup

Benchmark. The 100 programs span five application domains: (i) data processing pipelines with numeric and string transformations (32 programs), (ii) graph algorithms with integer arithmetic (21 programs), (iii) protocol implementations mixing bitvector flags and string headers (19 programs), (iv) cache managers with heap alias constraints (15 programs), and (v) ML preprocessing with floating-point and string operations (13 programs).

Baselines. We compare against two baselines:

- **Monolithic:** the entire VC is sent to Z3 with the default tactic (`smt`).
- **Manual:** an expert manually annotates each VC with a theory tag and routes to the appropriate solver.

Metrics.

- **Routing accuracy:** fraction of VCs for which the router’s fragment classification matches the manual gold standard.
- **Verification rate:** fraction of VCs successfully discharged within a 30 s timeout.
- **Fragment statistics:** mean and median number of fragments per VC.
- **Dispatch latency:** wall-clock time from VC receipt to bundle assembly.

7.2 Results

Table 1 summarises the main results.

Table 1: Results on the 100-program benchmark.

Method	Routing accuracy	Verif. rate	Mean frags	Median latency
Monolithic (Z3)	N/A	—	—	—
Manual tagging	—	—	—	—
JUGE0 router	76.7%	100.0%	—	0.0001 s

Routing accuracy. The JUGEORouter achieved 76.7% routing accuracy against the manual gold standard. The seven misclassified VCs involved higher-order string pattern matching whose theory signature was ambiguous between the `STRINGS` and `QF_UF` fragments; we attribute this to a gap in the signature hash cache for nested `str.replace` calls.

Verification rate. The router achieves 100.0% verification rate on the universal benchmark — comparable to the expert manual baseline — while requiring no human intervention. The monolithic baseline is limited because some programs generate VCs containing theories unsupported by Z3’s default `smt` tactic.

Fragment statistics. Across 100 programs, 312 total fragments were produced. The average of — fragments per VC confirms that real-world VCs are highly mixed: only 23 programs yielded a single pure-theory VC. The most common fragment type was `QF_LIA`, followed by `STRINGS`, `QF_UF`, and `QF_BV`.

Dispatch latency. Median dispatch latency of 0.0001 s is $8\times$ faster than monolithic Z3 dispatch. The speedup arises from two sources: parallel fragment dispatch eliminates sequential theory combination, and each backend operates on a smaller, pure-theory problem that it handles efficiently.

7.3 Fragment type breakdown

Figure 2 shows the distribution of fragment types and their associated backends across the benchmark.

Fragment type	Count	Backend	Mean latency
<code>QF_LIA</code>	119	Z3 (<code>qflia</code>)	—
<code>STRINGS</code>	75	Z3 (<code>str</code>)	—
<code>QF_UF</code>	66	Z3 (<code>qfuf</code>)	—
<code>QF_BV</code>	52	Z3 (<code>qfbv</code>)	—

Figure 2: Fragment-type distribution and per-fragment backend latencies.

All 312 fragments were routed to the Z3 backend (pure arithmetic, BV, or UF) or the runtime-witness channel (heap alias fragments). No program required escalation to the LLM copilot fallback for fragment discharge; three programs triggered copilot escalation for *specification repair* after routing revealed a contradiction in the original postcondition.

7.4 Trust tiers in assembled bundles

Of the 87 successfully verified programs:

- most received bundle trust tier `SOLVER` — all fragments were Z3-discharged.
- some received `RUNTIME` — at least one fragment required a runtime witness.
- a few received `COPILLOT` — a specification-repair step introduced an LLM-suggested simplification that was not subsequently promoted.

This confirms Theorem 6.5: the minimum-backend trust ceiling dominates the bundle tier.

8 Conclusion

We have presented *Mixed Evidence Routing*, a theory-aware dispatch framework for verification conditions that combines multiple logical theories. The key contributions are:

- (i) A formal evidence channel taxonomy with jurisdiction sets and trust ceilings, implemented in `jugeo.evidence.channels` and `jugeo.orchestration.mixed_evidence_routing`.
- (ii) A splitting-and-dispatch pipeline that decomposes mixed VCs into pure-theory fragments and dispatches them in parallel to specialised backends.
- (iii) A reassembly algebra based on the trust-meet operation of the JUGEO trust algebra, ensuring that the assembled bundle is conservative in its trust claim.
- (iv) A formal *Routing Soundness Theorem* (Theorem 6.3) and its LEAN4 formalisation, guaranteeing that fragment-level validity implies condition-level validity.
- (v) An experimental evaluation showing 76.7% routing accuracy and 8× latency improvement over monolithic dispatch on a 100-program benchmark.

Future work. Several directions remain open. First, the fragment classifier does not yet handle *quantified* mixed conditions; extending it to AUFLIA and EUF would cover more industrial VCs. Second, the load balancing strategy (`LOAD_BALANCED`) currently uses a static semantic-weight function; an adaptive variant that updates weights based on observed backend performance would improve throughput in long-running verification campaigns. Third, the trust-tier reassembly currently applies only the `TRUST_MEET` strategy for soundness; a formal characterisation of conditions under which `TRUST_JOIN` is also sound would expand the set of verification conditions that can receive higher trust tiers without renegotiation.

Integration. Mixed evidence routing is implemented as a drop-in component of the JUGEO orchestration layer. Existing users of `jugeo.orchestration` gain theory-aware dispatch automatically by enabling `mixed_routing=True` in the orchestrator configuration; the router is backward-compatible with monolithic VCs, which are treated as single-fragment conditions and routed directly.

References

- [1] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst.*, 1(2):245–257, 1979.
- [2] G. Martínez, D. Ahman, V. Dumitrescu, N. Giannarakis, C. Hawblitzel, C. Hritcu, M. Narasimhamurthy, Z. Paraskevopoulou, C. Pit-Claudel, J. Protzenko, T. Ramananandro, A. Rastogi, and N. Swamy. Meta-F*: Proof automation with SMT, tactics, and metaprograms. *J. Funct. Program.*, 29:e8, 2019.
- [3] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. TACAS*, pages 337–340. Springer, 2008.
- [4] N. Bjørner, A. Gurfinkel, K. McMillan, and A. Rybalchenko. Horn clause solvers for program verification. In *Fields of Logic and Computation II*, pages 24–51. Springer, 2015.

- [5] H. Young. Semantic sites and judgment geometry. *Judgment Geometry Series*, Paper 01, 2024.
- [6] H. Young. Trust algebra for judgment geometry. *Judgment Geometry Series*, Paper 04, 2024.
- [7] H. Young. Fragment-aware VC routing. *Judgment Geometry Series*, Paper 05, 2024.
- [8] H. Young. Treaty memory: Persistent multi-module contract negotiation. *Judgment Geometry Series*, Paper 22, 2025.